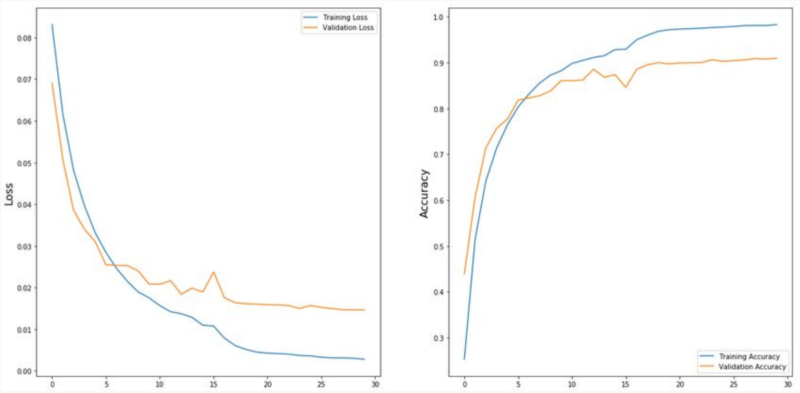
Wharton
University of Pennsylvania

Optimized Vision Transformer (ViT) Training Pipeline for CIFAR-10

Ayush Tripathi, Sai Minnal, Mayank Deoras

Problem Statement & Motivation

- Deep learning projects often optimize for final accuracy
- Final accuracy by itself doesn't fully represent efficiency
- Compute cost & hardware efficiency matter
  - Particularly with diminishing returns
- We are working to minimize time required for a ViT to reach 94% test accuracy on CIFAR-10



Standard accuracy and loss curves for model training against epochs — clear diminishing returns

Technical Approach: Objective + Evaluation Protocol

- Task: CIFAR-10 image classification

$$(x \in \mathbb{R}^{32\times32\times3}, y \in \{1, 2, ..., 10\})$$

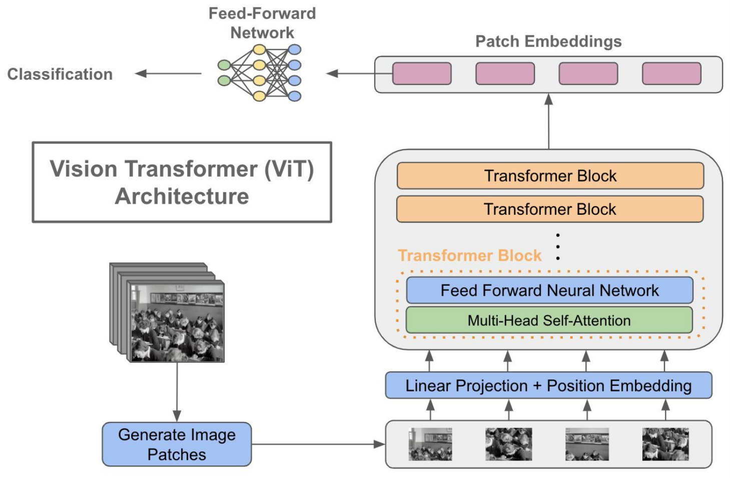
- Goal metric: time-to-target accuracy (target τ = 94%)

$$T_\tau(\theta) = \min\{t \geq 0 : A_{\text{test}}(t;\theta) \geq \tau\}$$

- Optimization Problem:

$$\theta^* = \arg\min_\theta T_\tau(\theta)$$

- What is θ — model hyperparameters and training method
- Metrics logged — wall-clock time, test accuracy each epoch, throughput (images/sec), peak GPU memory
    - Early stop once $A_{\text{test}} \geq \tau$



Standard Vision Transformer (ViT) architecture

Mathematical Formulation of Approach

- ViT breaks images into *N* patches (*P* by *P*) → feature vector (vector of length *d*)

$$N = \left(\frac{H}{P}\right)\left(\frac{W}{P}\right)$$

- ViT adds positional embeddings (attention doesn't know order/location)

$$Z_0 = [z_{\text{cls}}; z_1; ...; z_N] + E_{\text{pos}}$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- QK^T is an *N* by *N* matrix — attention cost grows like $O(N^2)$
    - Patch count *N* is a huge performance indicator

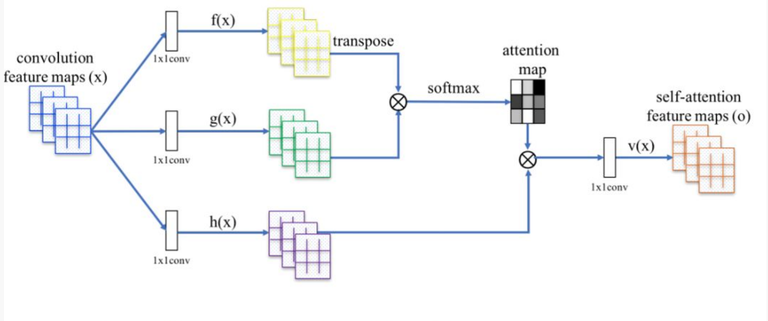


Illustration of the self-attention mechanism

Methodologies

Non-deterministic

- Training run isn't exactly reproducible each time
- Fixed seeds aren't used → enhanced training time

Automatic Mixed Precision (AMP)

- Uses lower precision arithmetic where safe to speed up computation and decrease memory usage

Random Augmentation

- Applied randomized image transformations during training
- Adds extra computer per step → increases data diversity

Learning Rate Schedulers

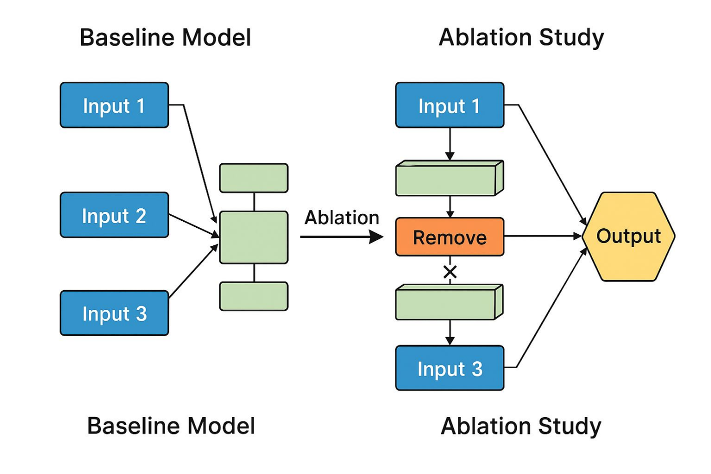
- Adjusts LR during training to improve time to convergence

Initial Results (using an A100 NVIDIA GPU)

Metric	Baseline Testing	Optimized Testing
Best Test Accuracy	94.1%	94.2%
Time to Target	205 sec	168 sec
Total Wall-Clock	320 sec	270 sec
Throughput	2,850 images/sec	3,450 images/sec
Peak GPU Memory	4.2 GB	3.1 GB

Next Steps

- Run ablation studies by isolating AMP, schedulers, and RandAugment
- Want to optimize GPU utilization
    - Optimize Tensor Core efficiency using AMP
- Compare deterministic vs nondeterministic modes
- Tune the learning rates and batch size under our optimized setup



Thank You!